

REVERSE ENGINEERING

BY 0X1D



CYBERSÉCURITÉ
AUDITS & FORMATIONS

FORMATEUR



Quentin Papelard

5 ans d'audits et formations
Des centaines d'applications Web
auditées

- **Formateur** cybersécurité pour les **services de renseignement**
- Ancien militaire (6 ans) : administration des **serveurs** les plus sensibles du gouvernement au service de la **présidence de la république** (**secret défense**)
- **Rigueur** et **pédagogie** au service des clients les plus exigeants
- La **confidentialité** comme seconde nature



OBJECTIFS

Vous donnez des bases (niveau CTF)

Vous donner les clés pour analyser un malware simple

Appliquer certaines mesures de sécurité linux

Introduire le module de sécurité shellcode



DÉROULÉ DE LA FORMATION

- Contexte, objectifs et prérequis
- Module 0: Installation des outils et ressources
- Module 1: Introduction au reverse engineering
- Module 2: Analyse statique
- Module 3: Analyse dynamique
- Module 4: Obfuscation et patching
- Module 5: Malware et vulnérabilités
- Module 6: TP noté (évaluation)



INSTALLATION DES OUTILS ET RESSOURCES



OUTIL : GHIDRA

- Outil créé par la NSA
 - Leak Vault7
- Installé par défaut sous Kali
- Sinon : <https://github.com/NationalSecurityAgency/ghidra>
- Nécessite Java
- Features : désassemblage, debugging, décompilation, cafetière



CRACKMES

- Ressources : Cf. Discord
- Décompresser l'archive



INTRODUCTION: CONTEXTE ET OBJECTIFS



OBJECTIFS

- Comprendre les analyses statique et dynamique
- Maîtriser désassemblage/décompilation
- Analyser un binaire inconnu, extraire comportements, détecter vulnérabilités
- Mettre en pratique via TP et évaluation finale
- Environnement: Linux (Ubuntu) recommandé, VM Kali isolée
- Éthique et cadre légal (France) respectés



PRÉREQUIS

- Prérequis: C/C++ de base, Linux CLI, notions d'ASM
- Outils: gdb, binutils, Ghidra/IDA Free, strace/ltrace



MODULE 1 — INTRODUCTION AU REVERSE ENGINEERING



M1 - OBJECTIFS

- Distinguer cas d'usage: malware, validation, CTF, audit
- Comprendre le cadre légal en France
- Revoir x86/x64: registres, pile, ABI
- Poser les bonnes pratiques de labo RE
- Préparer l'environnement outillé



M1 — CAS D'USAGE ET ÉTHIQUE

- Sécurité défensive: analyse d'incidents, IOC
- Validation logicielle: conformité, rétro-doc
- CTF/formation: apprentissage contrôlé
- R&D: interopérabilité (formats/protocoles)
- Interdit: contournement DRMs, usage non autorisé
- Toujours agir avec consentement explicite



M1 — CADRE LÉGAL (FRANCE)

- CPI L122-6-1: exceptions (interopérabilité, correction)
- L335-3: contrefaçon si hors exceptions
- RGPD / secret des affaires: prudence données
- Clause contractuelle > droit d'auteur
- Conservation d'artefacts: journaliser, horodater
- Traçabilité: notes, hash des fichiers

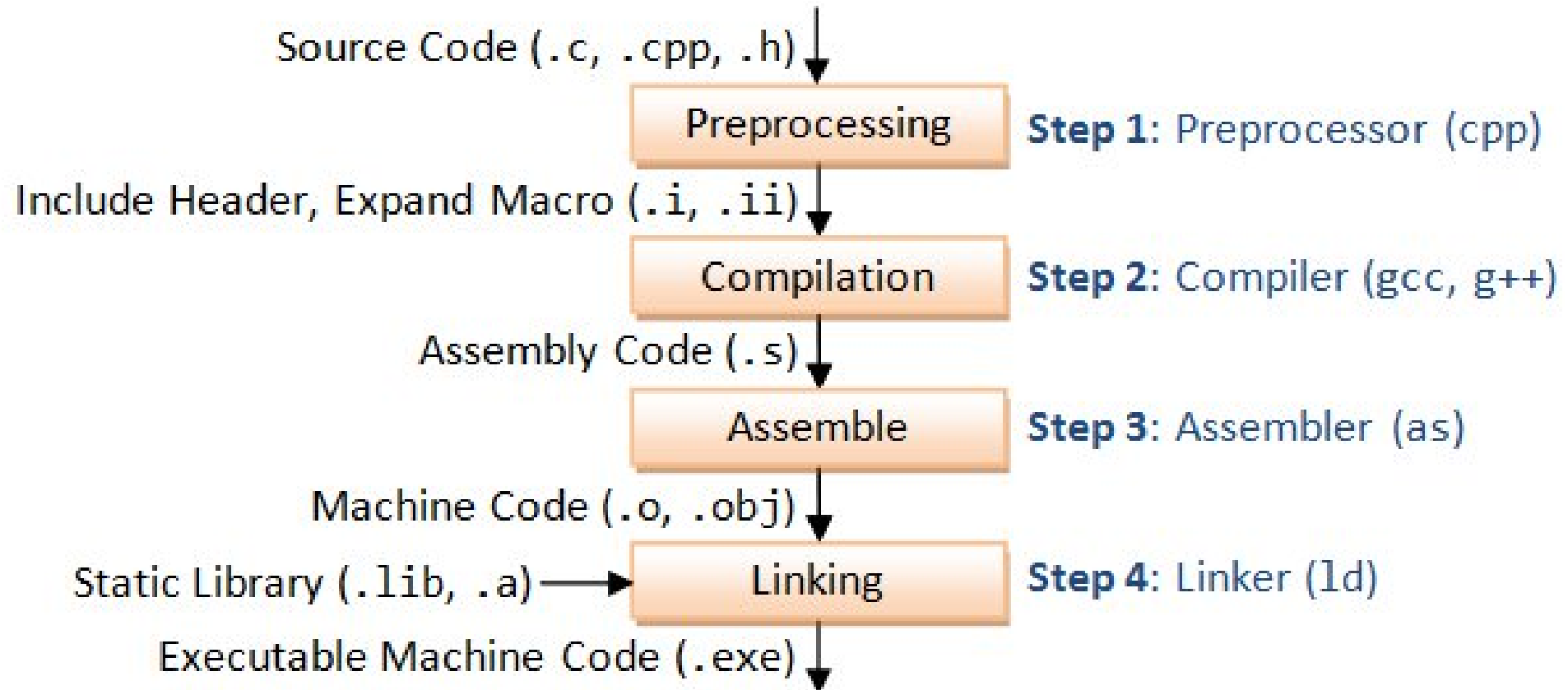


M1 — RAPPELS X86/X64: ARCHITECTURE

- Registres: x86 (EAX, EBX...), x64 (RAX... R15)
- Pile: croissance vers adresses basses, RSP/RBP
- Appels: conventions SysV AMD64 vs Windows
- Sections mémoire: .text, .data, .bss, heap, stack
- Instructions clés: mov, call, ret, cmp, jcc

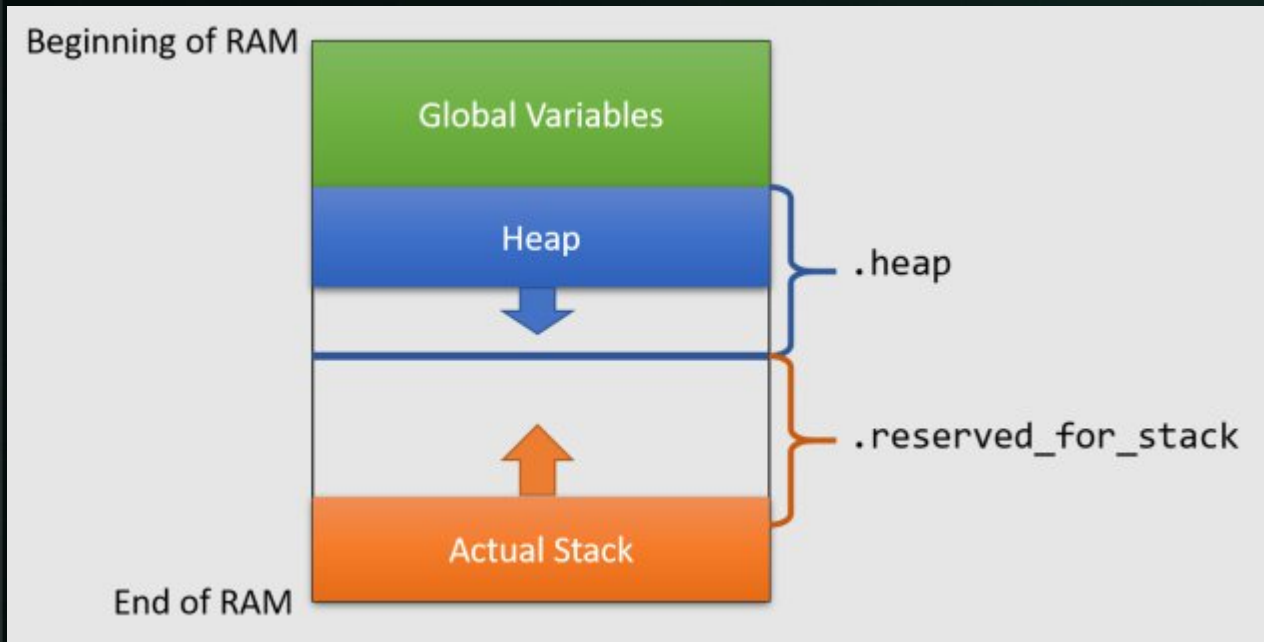


M1 — COMPILE ET ASSEMBLAGE





M1 — SEGMENTS MÉMOIRE



- `.text` : contient les instructions à exécuter
- `.data` : variables globales initialisées
- `.bss` : variables globales non-initialisées
- `Tas (Heap)` : variables via `malloc`
- `Pile (Stack)` : variables locales des fonctions, variables d'environnement



M1 — REGISTRES

name	64-bit	32-bit	16-bit	8-bit
rax	rax	eax	ax	ah / al
rbx	rbx	ebx	bx	bh / bl
rcx	rcx	ecx	cx	ch / cl
rdx	rdx	edx	dx	dh / dl
rsi	rsi	esi	si	sil
rdi	rdi	edi	di	dil
rbp	rbp	ebp	bp	bpl
rsp	rsp	esp	sp	spl
r8	r8	r8d	r8w	r8b
r9	r9	r9d	r9w	r9b
r10	r10	r10d	r10w	r10b
r11	r11	r11d	r11w	r11b
r12	r12	r12d	r12w	r12b
r13	r13	r13d	r13w	r13b
r14	r14	r14d	r14w	r14b
r15	r15	r15d	r15w	r15b

➤ Registres de données

➤ Registres de pile



M1 — INSTRUCTIONS ASSEMBLEUR

- `mov rbp, rsp`
 - Copie la valeur de RSP dans RBP
- `jmp RIP+4`
 - Exécute l'instruction se situant à l'adresse RIP +4 octets
- `call 0x0806ad5b`
 - Appelle la fonction située à l'adresse spécifiée
- `cmp RAX, RBX`
 - Compare les valeurs des registres
- `jne RIP+22`
 - Si les valeurs comparées sont différentes, sauter à l'instruction située à RIP+22
 - Voir aussi `je`, `jgt`, `jlt`, `jge`, `jle`, etc.
- `ret`
 - En fin de fonction, retourne à l'exécution avant appel de fonction
- `push rax`
 - Ajoute le contenu de RAX sur la pile
- `pop rax`
 - Retire 64 bits de données du sommet de la pile et les stocke dans RAX



M1 — FLUX D'EXÉCUTION ET ABI

- Prologue/épilogue
- Passage d'arguments
- Retour
- Alignement de pile
- Appels indirects et PLT/GOT



M1 — PROLOGUE ET ÉPILOGUE

➤ Instructions appelées avant chaque appel de fonction

`push rbp` ; sauvegarde l'ancien pointeur de base

`mov rbp, rsp` ; rbp pointe maintenant sur le début de la pile de la fonction

➤ Instructions appelées au retour de la fonction

`mov rsp, rbp` ; restaure la pile

`pop rbp` ; récupère l'ancien pointeur de base

`ret` ; retourne à l'appelant



M1 — PILE

STACK		
Address	Memory	Register
0x00	-----	<-- RSP
	Local variable 3	
	Local variable 2	
	Local variable 1	<-- RBP
	Saved base pointer	
	Return address	
	argv[0]	
0xFF	argv[1]	
	argv[2]	

➤ Appel de fonction :

- Les paramètres de la fonction sont empilés
- Puis l'adresse de retour (valeur RIP à charger lors de l'appel à RET)
- Puis le RBP actuel
- RBP = RSP

➤ Les variables locales sont empilées

- RSP est incrémenté à chaque fois



M1 — PASSAGE D'ARGUMENTS ET RETOUR DE FONCTION

- Les premiers arguments sont stockés dans les registres suivants :

rdi, rsi, rdx, rcx, r8, r9

- Si la fonction nécessite davantage d'arguments : ils vont dans la pile
- Lorsque la fonction retourne une valeur, celle-ci est stockée dans RAX.



M1 — ALIGNEMENT DE PILE

- RSP configuré pour maintenir un alignement de 16 octets
- Evite les fautes d'alignement
- Instructions caractéristiques :
sub rsp, 0x20 ; réserve 32 octets, maintient l'alignement



M1 — APPELS INDIRECTS

- **Appel direct** : `call <adresse>` → cible connue au moment de la compilation
- **Appel indirect** : `call rax` ou `call [rip+offset]` → cible déterminée à l'exécution
 - Utilisé pour : pointeurs de fonction, tables virtuelles, résolutions dynamiques
- **PLT (Procedure Linkage Table)** :
 - Pont entre le code et les bibliothèques dynamiques
 - Contient des stubs appelant les fonctions externes
- **GOT (Global Offset Table)** :
 - Table contenant les adresses réelles des fonctions/imports résolus à l'exécution
- **Astuce Reverse engineering** :
 - Repérer un `call` vers la PLT indique un appel à une fonction de bibliothèque (ex. `printf@plt`)
 - Suivre la GOT permet de voir où est résolue l'adresse effective



M1 — INSTALLATION ET CONFIGURATION



M1 — INSTALLATION ET CONFIGURATION

➤ Paquets de base (Debian/Ubuntu)

```
sudo apt update
```

```
sudo apt install build-essential gdb binutils \  
    strace ltrace file python3 python3-pip
```

➤ Extensions gdb (option): gef ou pwndbg

➤ Ghidra (dernière version)



TP - PRISE EN MAIN ELF (1/2)



- Sujet : executable hello
- Identifier l'architecture avec file
- Lister sections avec readelf -S
- Localiser main via nm/objdump -d
- Repérer l'appel printf et sa résolution PLT



TP - PRISE EN MAIN ELF (2/2)



- Importer hello dans Ghidra
- Lancer l'analyse auto, ouvrir la fonction main
- Prise en main ASM/Décompilateur



M1 — RÉSUMÉ

- Cas d'usage légitimes et limites légales
 - Nécessite l'accord de l'éditeur
- Notions x86/x64 et ABI SysV
 - X64 universel de nos jours
- Outils de base: strings, objdump, readelf
- Premiers pas Ghidra
- Journalisation et bonnes pratiques de labo
 - Ne pas hésiter à utiliser votre logiciel de prise de notes préféré (Obsidian, etc.)



MODULE 2 — ANALYSE STATIQUE



M2 — ANALYSE STATIQUE : OBJECTIFS

- Formats ELF/PE et symboles
- Maîtriser binutils: readelf, objdump, strings
- Utiliser Ghidra pour cartographier
- Identifier fonctions, chaînes, références croisées



M2 — ELF STRUCTURE

- En-tête, Program Headers, Sections
- Sections clés: .text, .rodata, .plt, .got
- Tables: symtab/dynsym, relocations
- https://upload.wikimedia.org/wikipedia/commons/e/e4/ELF_Executable_and_Linkable_Format_diagram_by_Ange_Albertini.png

```
~$ uname -m
armv7l
~$ ./simple.ARM
Hello World!
```

```
000: 7F 45 4C 46 01 01 01 00 00 00 00 00 00 00 00 00 .ELF.....
02 00 28 00 01 00 00 00 60 00 00 00 40 00 00 00 ..(.....~.@...
B0 00 00 00 00 00 00 00 34 00 20 00 01 00 28 00 .....4.....
04 00 03 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
040: 01 00 00 00 00 00 00 00 00 00 00 00 08 00 00 00 .....
90 00 00 00 90 00 00 00 05 00 00 00 00 00 00 00 .....
060: 0D 20 A0 E3 14 10 8F E2 01 00 A0 E3 04 70 A0 E3 .....p..
00 .....
080: 48 .....fi....
090: 00 .....text
00 .....
0B0: SHA-1 4709F4A546C384D946E9CF048C1DD07D5D735592
00 DOWNLOAD @ ELF101.CORKAM1.COM
00 .....
00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00 .....
06 00 00 00 60 00 00 00 60 00 00 00 20 00 00 00 .....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
11 00 00 00 01 00 00 00 02 00 00 00 00 00 00 00 .....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00 00 00 00 00 00 00 01 00 00 00 00 03 00 00 00 .....
00 00 00 00 00 00 00 90 00 00 00 19 00 00 00 .....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

SIMPLE.ARM

7F 45 4C
02 00 28
B0 00 00
04 00 03

HEADER^{1/2}

TECHNICAL DETAILS FOR
IDENTIFICATION AND EXECUTION

01 00 00
90 00 00

0D 20 A0
00 00 00

SECTIONS

CONTENTS OF THE EXECUTABLE

48 65 6C

00 2E 73
00 2E 72

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00 00 00
00 00 00
06 00 00
00 00 00
11 00 00
00 00 00
00 00 00
00 00 00
00 00 00

HEADER^{2/2}

TECHNICAL DETAILS FOR LINKING
(IGNORED FOR EXECUTION)

00 00 00
00 00 00
06 00 00
00 00 00
11 00 00
00 00 00
00 00 00
00 00 00
00 00 00

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

7F 45 4C 46 01 01 0
02 00 28 00 01 00 0
B0 00 00 00 00 00 0
04 00 03 00 00 00 0

ELF HEADER

IDENTIFY AS AN ELF TYPE
SPECIFY THE ARCHITECTURE

0 .ELF.....
0 ..(.....~.@...
04.....
0

01 00 00
90 00 00

PROGRAM HEADER TABLE

EXECUTION INFORMATION

0D 20 A0 E3 14 10 8F
00 00 00 EF 01 00 A0

CODE

EXECUTABLE INFORMATION

E3p
EFp

48 65 6C 6C 6F 20 57

DATA

INFORMATION USED BY THE CODE

00 Hello.World!..

00 2E 73 68 73 7
00 2E 72 6F 64 6

SECTIONS' NAMES

..shstrtab..te
..rodata.....

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00

06 00 00 00 60 00 00 00 60 00 00 00 20 00 00 00

00 00 00

11 00 00

80 00 00

00 00 00 00 00 00 00 00 01 00 00 00 03 00 00 00

00 00 00 00 00 00 00 00 90 00 00 00 19 00 00 00

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

SECTION HEADER TABLE

LINKING (CONNECTING PROGRAM OBJECTS) INFORMATION

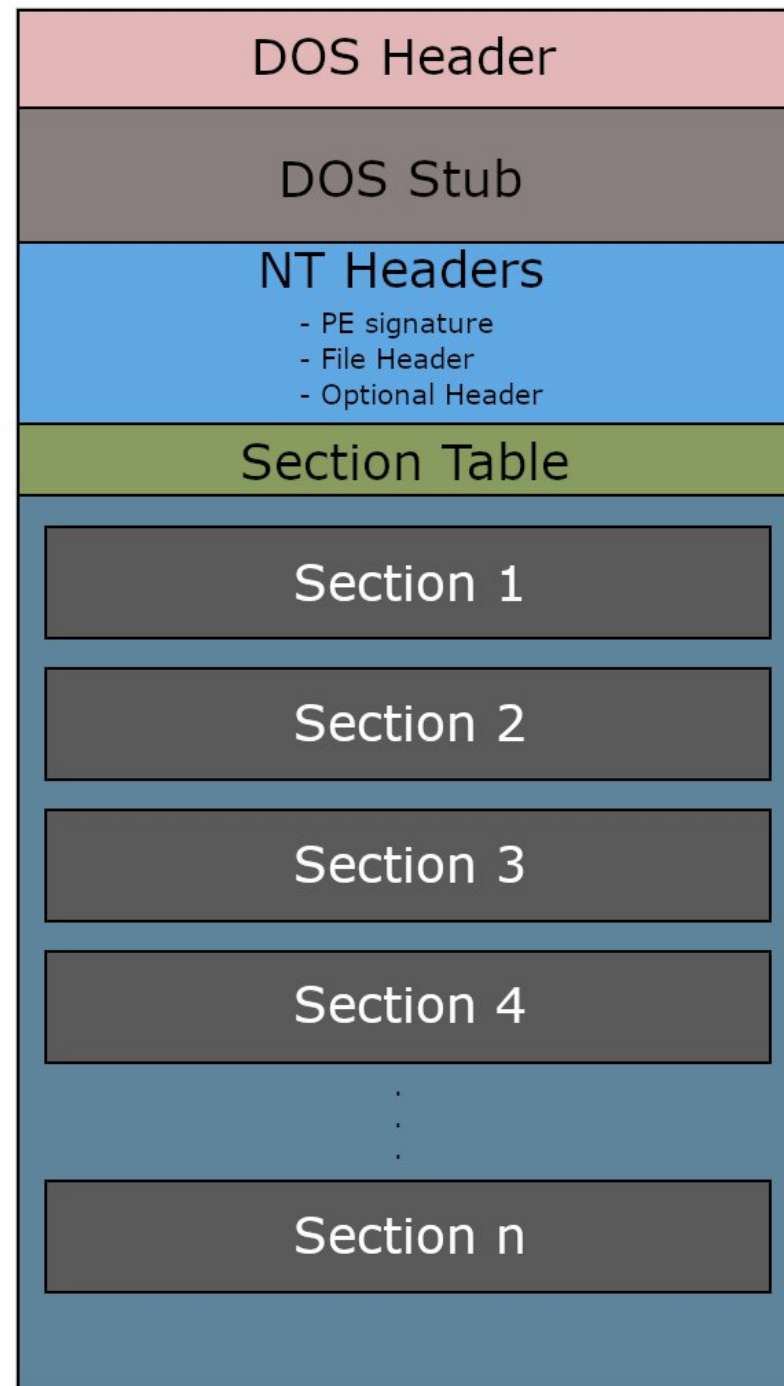


M2 — PE STRUCTURE

➤ Portable Executable

- Format des programmes et DLL sous Windows
- Hérité du format COFF (Common Object File Format)
- Utilisé pour .exe, .dll, .sys, .ocx, etc.
- Composé d'en-têtes + sections + tables (imports/exports...)
 - Table d'import : Contient la liste des bibliothèques externes utilisées (DLL)
 - Table d'export : Présente dans les DLL (et certains EXE), liste des fonctions et variables exportées vers d'autres modules

M2 — PE STRUCTURE





M2 — PE STRUCTURE - OUTILS

➤ pe-tree

```
git clone https://github.com/blackberry/pe\_tree.git  
cd pe_tree  
python3 -m venv env  
source ./env/bin/activate  
pip install -e .
```



M2 — SYMBOLES ET RELOCATION

- Symboles: fonctions, objets, portée
- Relocations: résolution d'adresses dynamiques
- PLT/GOT: mécanisme d'import lazy
- Stripping: impact sur analyse statique
- Heuristiques sans symboles (FLIRT, sigs)

M2 — SYMBOLES: FONCTIONS, OBJETS, PORTÉE



➤ Qu'est-ce qu'un symbole ?

- Nom logique associé à une adresse ou une donnée dans le binaire.
- Peut représenter :
 - Une fonction (code exécutable)
 - Un objet (variable globale ou statique)

➤ Portée (Scope) :

- Globale : visible par d'autres modules / binaire
- Locale : visible uniquement dans le module courant



M2 — RELOCATIONS : RÉOLUTION D'ADRESSES DYNAMIQUES

- **But** : ajuster les adresses lors du chargement en mémoire
- **Nécessaire pour** :
 - Les bibliothèques partagées (position indépendante)
 - Les binaires PIE (Position Independent Executable)
- **Mécanisme** :
 - Le compilateur/linker insère des entrées de relocation.
 - Le loader de l'OS applique les offsets réels à l'exécution.

M2 — PLT / GOT : MÉCANISME D'IMPORT « LAZY »



➤ GOT (Global Offset Table) :

- Table contenant les adresses réelles des symboles importés.
- Initialement remplies par des pointeurs vers des stubs.

➤ PLT (Procedure Linkage Table) :

- Stubs qui appellent la routine de résolution dynamique (ld.so).
- Premier appel → résout l'adresse réelle, met à jour la GOT.
- Appels suivants → sautent directement à l'adresse résolue (lazy binding).



M2 — STRIPPING : IMPACT SUR L'ANALYSE STATIQUE

- **Stripping** = suppression des symboles du binaire (avec strip)
- **Effets :**
 - Taille plus petite
 - Masque les noms de fonctions, variables, sections inutilisées
- **Conséquence RE :**
 - Le désassemblage devient anonyme (sub_401000, func_1234...)
 - Perte de contexte, analyse plus difficile
- **Astuce :**
 - Utiliser la version non-strippée pour développer/debugger ; strippée pour distribuer.



M2 — HEURISTIQUES SANS SYMBOLES (FLIRT, SIGNATURES)

- **Problème** : binaire strippé → pas de noms, pas de symboles
- **Solution** : heuristiques et signatures :
 - **FLIRT** (Fast Library Identification and Recognition Technology) d'IDA
 - **Signatures** de fonctions standard (hash, motifs de bytes)
 - Reconnaissance d'empreintes de librairies
- **Avantages** :
 - Restaure des noms connus (printf, malloc...) même dans un binaire strippé
 - Accélère l'analyse statique
- **Limites** :
 - Fonctionne surtout sur du code de librairies connues
 - Les fonctions custom restent anonymes



M2 — POINTS CLÉS À RETENIR

- Les **symboles** facilitent énormément l'analyse ; sans eux, on passe par des heuristiques.
- Les **relocations** et **PLT/GOT** sont au cœur du chargement dynamique et de l'import lazy.
- Le **stripping** complique volontairement l'analyse statique.
- Des outils comme **FLIRT** ou des bases de signatures peuvent « redonner » des noms aux fonctions.



M2 — FLUX DE CONTRÔLE ET CFG

- CFG = Control Flow Graph
- Fonctions et graphes d'appel
- Branches conditionnelles et indirectes
- Tables de dispatch (switch)
- Inlining et optimisation: effets
- Indicateurs de complexité (cyclomatique)



M2 — FONCTIONS ET GRAPHE D'APPEL

- **Fonctions** : unités de code avec un point d'entrée et un point de sortie
- **Graphe d'appel (Call Graph)** :
 - Noeuds = fonctions
 - Arêtes = appels entre fonctions
- **Utilité en RE** :
 - Visualiser les dépendances entre modules
 - Identifier les fonctions critiques, les points d'entrée externes
- **Outils** :
 - IDA/Ghidra : vue graphique des appels
 - call direct vs call indirect



M2 — BRANCHES CONDITIONNELLES ET INDIRECTES

- **Branches conditionnelles** : if/else, while, for
 - Se traduisent en instructions de saut conditionnel (jz, jnz, ja, jb...)
 - Génèrent plusieurs chemins dans le CFG
- **Branches indirectes** : saut dont la cible est contenue dans un registre ou une variable
 - jmp rax, call [rbx+offset]
 - Utilisées pour tables virtuelles, callbacks, obfuscation
- **Impact RE** :
 - Les branches indirectes compliquent la reconstruction du CFG
 - Analyse statique doit résoudre les cibles possibles



M2 — TABLES DE DISPATCH (SWITCH)

- **Switch-case en C/C++ :**
 - Génère souvent une **jump table** (table de dispatch)
 - Table = adresses des branches, indexée par la valeur du switch
- **Dans le binaire :**
 - On voit un calcul d'offset et un saut indirect vers `table[index]`



M2 — INLINING ET OPTIMISATION : EFFETS SUR LE CFG

- **Inlining** : le compilateur insère le corps d'une fonction à l'endroit de l'appel
 - Réduit l'overhead d'appel
 - Élimine parfois la fonction originale
- **Autres optimisations** : suppression de branches, fusion de boucles...
- **Conséquences pour l'analyste** :
 - Les graphes d'appel perdent des noeuds (fonctions « disparues »)
 - CFG plus complexe, moins lisible
 - Les signatures de fonctions ne correspondent plus exactement



M2 — INDICATEURS DE COMPLEXITÉ (CYCLOMATIQUE)

➤ Complexité cyclomatique (McCabe) :

- Mesure le nombre de chemins indépendants dans un programme
- Formule : $M = E - N + 2P$ (E=arêtes, N=noeuds, P=composants connexes)

➤ Intérêt :

- Évaluer la difficulté de test et de maintenance
- En RE : repérer les fonctions les plus complexes à analyser

➤ Dans les outils :

- IDA/Ghidra affichent souvent un nombre d'arcs/noeuds dans la vue CFG
- Peut guider l'effort d'analyse



M2 — POINTS CLÉS À RETENIR

- **CFG** = représentation des flux possibles d'un programme
- Branches conditionnelles & indirectes définissent les chemins
- Jump tables et dispatch montrent les structures de contrôle avancées
- Inlining & optimisations modifient le graphe d'appel et compliquent l'analyse
- Complexité cyclomatique aide à prioriser l'effort de reverse engineering



M2 — DÉCOMPIATION: LIMITES

- Pseudocode vs source original
- Variables et types inférés
- Artefacts d'optimisation (O2/O3)
- Obfuscation: patterns usuels
- Vérification croisée ASM $\leftrightarrow$ pseudo



M2 — PSEUDOCODE VS. SOURCE ORIGINAL

➤ **Décompilation $\neq$ code source**

- La décompilation produit un **pseudocode** approximatif
- Pas de commentaires, pas de noms de variables, pas de structures exactes

➤ **Différences clés :**

- Style et indentation générés par l'outil
- Reconstruction approximative des structures de contrôle
- Macros, inline assembly, directives du compilateur perdues



M2 — VARIABLES ET TYPES INFÉRÉS

- Les décompilateurs **devinent** :
 - Les types (int, char*, struct...) à partir des opérations et accès mémoire
 - Les noms de variables (var_1, local_2...) arbitraires
- Conséquences :
 - Risque d'erreur dans les types (int vs unsigned, pointeurs vs tableaux)
 - Structures complexes mal reconstruites
- **Bonne pratique** :
 - Toujours vérifier les types inférés avec le désassemblage



M2 — ARTEFACTS D'OPTIMISATION (02/03)

- Les options d'optimisation du compilateur transforment fortement le code :
 - Inlining de fonctions
 - Suppression de variables temporaires
 - Fusion ou réécriture de boucles et branches
- Impact sur le pseudocode :
 - Fonctions « disparues »
 - Logique réarrangée ou non reconnaissable
 - CFG plus plat ou plus éclaté
- **Analyste RE** : garder à l'esprit que le code affiché n'est pas « écrit » ainsi par le développeur



M2 — OBFUSCATION : PATTERNS USUELS

- **Obfuscation volontaire** pour compliquer la rétro-ingénierie :
 - Contrôle de flux artificiel (sauts inutiles, opaque predicates)
 - Variables globales réutilisées, chiffrement des chaînes
 - Appels indirects, dispatch par tables
- **Effets dans le pseudocode :**
 - Fonctions énormes et illisibles
 - Noms et structures incohérents
 - Sauts non résolus ou nombreux labels
- **Approche :**
 - Revenir au désassemblage pour comprendre les blocs réels



M2 — VÉRIFICATION CROISÉE ASM ↔ PSEUDOCODE

- **Toujours recouper le pseudocode avec l'assembleur :**
 - Le pseudocode est une hypothèse, l'ASM est la réalité
 - Identifier les instructions critiques (syscalls, accès mémoire)
 - Comprendre la pile et les registres pour valider les arguments
- **Avantage :**
 - Détecter les faux positifs du décompilateur
 - Reconstituer correctement la logique réelle



M2 — POINTS CLÉS À RETENIR

- La décompilation donne un **aperçu pratique** mais jamais le code source exact.
- Types, variables et structures sont **devinés**, donc potentiellement faux.
- Les **optimisations** et l'**obfuscation** dégradent encore la fidélité du pseudocode.
- Le **désassemblage** reste la **référence** : toujours vérifier dans l'ASM ce que suggère le pseudocode.



TP - PREMIER CRACKME 1 / 2



- Ouvrir license_check dans Ghidra
- Localiser les chaînes “LICENSE” / “VALID” et toute chaîne pertinente
- Reconstituer l’algorithme (hash, somme, etc.)
- Découvrir la clé de licence



TP - PREMIER CRACKME 2/2



- Écrire un script Python qui génère une clé
- Tester le binaire avec votre sortie
- Déterminer les limites du keygen
- Proposer une contre-mesure côté dev

M2 — RÉSUMÉ



- Lecture approfondie ELF/PE
- Symboles, relocations, PLT/GOT
- Ghidra/IDA: flux et références
- Méthodologie pour reconstituer algorithmes
- Limites de la décompilation



MODULE 3 — ANALYSE DYNAMIQUE



MODULE 3 — OBJECTIFS

- Débuguer avec gdb (+gef/pwndbg)
- Tracer syscalls et libc avec strace/ltrace
- Hooking contrôlé via LD_PRELOAD
- Capturer comportements à l'exécution



M3 — GDB

- Debugger en ligne de commande
- Très utilisé sous linux
- Syntaxe : `gdb ./license_check`
 - `run param1 param2 ...`
- Commandes clés :
 - `run, continue, exit`
 - `break, watch`
 - `next, ni, step, si`
 - `info registers`
 - `p/x $rax`
 - `set $rip = 0x401234`
 - `bt`



M3 — PROCESSUS, THREADS, SIGNAUX — CONCEPTS CLÉS

- Processus : instance d'un binaire avec espace d'adressage, descripteurs, contexte d'exécution.
 - PID, espace mémoire (code/data/heap/stack/mmap), tables FD
- Threads : unités d'exécution partageant l'espace mémoire du processus (TID/LWP).
 - Concurrency : races, deadlocks, breakpoints multi-threads
- Signaux : interruptions logicielles (SIGINT, SIGSEGV, SIGTRAP, SIGCHLD...).
- Handlers (signal, sigaction), masques, dispositions
- ptrace : API noyau utilisée par les debuggers (GDB/LLDB) pour stopper/inspecter/modifier.



M3 — BREAKPOINTS

➤ Breakpoints

➤ **Logiciels** : remplacent 1 octet d'instruction par un trap (ex : 0xCC en x86).

➤ **Matériels (HW)** : s'appuient sur registres de debug CPU (DR0-DR7 en x86) → limites de nombre, plus fiables.

➤ Dans GDB :

➤ **Fonction/ligne** : `break main`, `break file.c:42`

➤ **Adresse** : `break *0x40123a`

➤ **Conditionnels** : `break foo if x==0xdeadbeef`

➤ **Temporaires** : `tbreak` (se désarment après 1 hit)

➤ `break check_license`



M3 — WATCHPOINTS (SUR DONNÉES)

- HW watchpoints : surveillent une adresse/zone en R/W/X sans patcher le code.
- Idéal pour : « qui modifie cette variable/buffer ? »
- Dans GDB :
 - `watch *0x7fff7dd3000 # write-watchpoint`
 - `info watchpoints`
- `watch license`



M3 — STEPPING & CONTRÔLE D'EXÉCUTION

- Exécuter : `run [args]`, `start`
- Continue : continue (jusqu'au prochain arrêt)
- Step vs Next :
 - `step/si` : entre dans les appels
 - `next/ni` : saute par-dessus
- Finish/Until : sortir de la fonction / avancer jusqu'à une ligne/adresse
- Threading :
 - `info threads`, `thread <n>`
 - `set schedule-multiple on` pour laisser faire plusieurs threads entre BP



TP - BUFFER OVERFLOW



- Lancer le programme « bof » sous GDB
 - Tester sans paramètre, puis avec une chaîne de plus en plus longue jusqu'au crash du programme
 - Inspecter via gdb pour déterminer ce qui s'est produit
- Pistes :
 - Watchpoint sur buf
 - Taille de buf VS taille de la chaîne fournie
 - Garder un œil sur RIP



M3 — EXAMPLES: STRACE/LTRACE



➤ Observer fichiers et réseau

```
strace -f -o trace.txt ./mystery ABC
```

```
ltrace -o ltrace.txt ./mystery ABC
```

```
grep -E "open|connect" trace.txt
```



MODULE 4 — OBFUSCATION ET PATCHING: OBJECTIFS



MODULE 4 — OBFUSCATION ET PATCHING: OBJECTIFS

- Reconnaître packing/obfuscation
- Détecter anti-debug simples
- Réaliser un patch binaire localisé
- Pratiquer sur crackmes de labo



M4 — PACKING ET DÉTECTION



M4 — PACKING ET DÉTECTION

➤ Objectifs

- Identifier rapidement un binaire **packé** (UPX et autres)
- Connaître les **indicateurs** : entropie, sections, imports
- Utiliser des **outils** (file, diéc, PEiD, capa) et des **heuristiques**
- Savoir **dépacker** (UPX) ou mettre en place un **unpack dynamique**



M4 — PACKING

➤ UPX & packers courants

- UPX (Unix/Windows) : très répandu, compression LZMA/UCL, facile à dépacker
 - Signatures : section/segment UPX (UPX0, UPX1, UPX2), file → "UPX compressed"
 - Dépack/repacks simples : `upx -d sample` / `upx -9 sample`

➤ Packers Windows courants

- ASPack, FSG, MPRESS, Themida/WinLicense (protections → anti-debug/VM), UPX modifié

➤ Packers Linux/ELF

- UPX, MPRESS ELF, Shc, Packer custom (malware)



M4 — DÉTECTION PAR ENTROPIE

➤ Entropie élevée, sections atypiques

➤ **Entropie** : proche de 8 bits/byte → suspicion de compression/chiffrement

➤ Mesure rapide : `binwalk -E sample` · `rabin2 -EE sample (radare2)` · `diac -n`

➤ Sections/segments atypiques

➤ Windows/PE : noms non standard (UPX0/1, .boom, .aspack), peu de sections, SizeOfRawData vs VirtualSize incohérents

➤ Linux/ELF : segments PT_LOAD compressés, .upx*, .text minuscule, grosses .data/.rdata, absence de symboles

➤ **Entrypoint** pointant vers une zone compressée/stub packer (sauts, décompression en mémoire)

M4 — DÉTECTION : HEURISTIQUES ET SIGNATURES



➤ Heuristiques vs signatures

➤ Signatures (PEiD/DIE) :

- Rapides, déterministes pour packers connus
- Mais faciles à contourner (mutations, rebasing, renommage sections)

➤ Heuristiques (entropie, structure PE/ELF, imports, comportement runtime) :

- Généralisables aux packers inconnus/custom
- Mais faux positifs possibles, demandent corrélation



M4 — AUTRES MÉTHODES DE DÉTECTION

- **Vérifications : file, DIE, PEiD (Windows)**
 - **file** (cross-platform) : détecte certains UPX/packers
 - **DIE / Detect It Easy** (GUI & CLI diec) : signatures + entropie + PE/ELF infos
 - **PEiD** (legacy, Windows) : base de signatures packers/protecteurs
 - **Exeinfo PE, RDG Packer Detector** : alternatives/compléments
 - **capa (Mandiant)** : détection capacités (heuristique par features) utile après dépack

M4 — MÉTHODES DE DEPACKING



➤ Stratégies d'unpack

- **Statique** : si packer connu (UPX/MPRESS) → outil natif/depacker tierce
- **Dynamique** : exécuter sous debugger, **break** après la décompression puis **dumpez** l'image en mémoire
 - Windows : x32dbg/x64dbg → BP sur VirtualAlloc, VirtualProtect, OEP heuristique
 - Linux : gdb + break sur mmap, mprotect, execve ; dump /proc/<pid>/mem ou via gcore



M4 — APPROCHE RECOMMANDÉE

- Regarder **entropie & sections** (binwalk/rabin2)
- Inspecter **imports/IAT** (lief, pev, dumpbin) → minimales ?
- Exécuter en **sandbox/debug** pour détecter l'**unpack** (APIs mémoire + OEP)
- Dump l'image mémoire, **reconstruire imports**, ré-analyser (capa/strings/ghidra)



M4 — ANTI-DEBUG BASIQUES



M4 — ANTI-DEBUG BASIQUES

➤ Objectifs

- Reconnaître les **techniques anti-debug** courantes
- Savoir **reproduire et contourner proprement** (éthique)
- Préparer un binaire au **debug pédagogique** (neutralisation ciblée)



M4 — PTRACE_TRACEME : PRINCIPE

➤ **Principe** : le processus se déclare « déjà tracé » → empêche un debugger de s'attacher.

➤ **Détection alternative** : lire /proc/self/status → TracerPid: 0 attendu.

```
#include <sys/ptrace.h>
#include <stdio.h>
#include <errno.h>
int anti_debug(void){
    errno = 0;
    long r = ptrace(PTRACE_TRACEME, 0, NULL, NULL);
    if (r == -1 && errno) return 1; // suspect: déjà tracé
    return 0;
}
```

```
int tracerpid(void){
    FILE *f=fopen("/proc/self/status","r");
    if(!f) return -1;

    char buf[512];

    while(fgets(buf,sizeof(buf),f)) {
        if(!strncmp(buf,"TracerPid:",10))
            return atoi(buf+10);
    }
    return -1;
}
```



M4 — PTRACE_TRACEME : CONTOURNEMENT

- **GDB** : breakpoint avant ptrace, forcer le retour à -1 et errno=EPERM.

```
# 1) Neutraliser ptrace() à la volée
(gdb) break ptrace
(gdb) commands
> set $rax=-1; set errno=1; c
> end
```



M4 — GETPPID, TIMING (RDTSC, SLEEP)

➤ **Idée** : détecter un parent inattendu (debugger) et/ou mesurer des lenteurs dues au pas-à-pas.

```
#include <time.h>
#include <unistd.h>
#include <x86intrin.h>
int timing_probe(){
    unsigned long long t0=__rdtsc();
    for (volatile int i=0;i<1000000;i++);
    unsigned long long dt=__rdtsc()-t0;
    return (dt > 2000000ULL); // heuristique
}
```



M4 — GETPPID, TIMING : CONTOURNEMENT

- **Exécution non pas-à-pas :**
préférer breakpoints bien placés
+ finish/until



M4 — ISATTY

- **Heuristiques** : refuser de s'exécuter si stdin/stdout **pas TTY**, ou si variables **suspectes** sont présentes.

```
#include <unistd.h>
#include <stdlib.h>
int env_checks(){
    if(!isatty(0) || getenv("LD_PRELOAD")) return 1; // suspect
    return 0;
}
```

- **Contournement:**

- Lancer avec un **pseudo-terminal** :
 - `script -q -c "./prog" /dev/null`
 - `socat -d -d`
`exec:./prog,pty,setsid,ctty`
- Nettoyer l'env :
 - `env -u LD_PRELOAD ./prog`
- **Hooker** `getenv` / `isatty` pour valeurs « saines » (en lab uniquement)





M4 — DÉTECTION DE BREAKPOINTS

- Scan code pour octet 0xCC (x86 INT3) ou checksum du texte
- Lecture de la **page exécutable** et vérification de l'intégrité
- **Contournement:**
 - Utiliser des **HW breakpoints** (watchpoints DRx) → pas de patch 0xCC
 - Break sur **API** adjacentes (ex. malloc, read) ou sur **branches** (break conditionnels)
 - **Patcher** runtime le binaire pour **désactiver** le scan (en environnement de lab)

```
int has_int3(const unsigned char* p, size_t n){  
    for (size_t i=0;i<n;i++) if (p[i]==0xCC) return 1; return 0;  
}
```

TP - UNTRACEABLE BINARY



- Tenter de lancer le programme « untraceable » dans GDB.
- Contourner les mesures en place afin de réussir à debug le programme.



M4 — PATCHING: PRINCIPES



M4 — PATCHING: PRINCIPES

➤ Objectifs

- Neutraliser un **check** via **NOP-sled**
- **Inverser un branchement** (JNZ→JZ / JNE→JE)
- **Rediriger l'exécution** via un **JMP court**
- Méthodo propre : **sauvegarde, hashing, traçabilité**



M4 — PATCHING: WORKFLOW

- Sauvegarder et hasher l'original
- Localiser la condition / la protection
- Décider du patch (nop ? Inversion ?
 Jump ?)
- Assembler les octets via nasm
- Patcher le fichier
- Re-hasher, faire le diff

```
cp --reflink=auto sample.bin sample.orig  
sha256sum sample.bin > sample.bin.sha256  
file sample.bin; checksec --file sample.bin || true
```

```
objdump -Mintel -d sample.bin | less
```



M4 — PATCHING: NOP SLED

➤ **Cas d'usage** : supprimer un test/licence/anti-debug simple sans casser l'alignement.

➤ **Exemple de protection:**

```
cmp eax, 1
```

```
jne fail
```

➤ Remplacer cmp;jne par NOP (0x90) de même longueur

➤ = Instruction vide

➤ Déterminer l'adresse de l'instruction

➤ Modifier le binaire

➤ `printf "\x90\x90" | dd of=sample.bin bs=1 seek=$((0x401235)) conv=notrunc`

➤ Il faut autant de \x90 que d'octets à écraser



M4 — PATCHING: INVERSION DE CONDITION

➤ **Principe** : basculer la logique sans toucher aux comparaisons.

➤ JNZ/JNE courte (8-bit rel) : opcode 0x75 → JE/JZ courte : 0x74

➤ JNZ near (32-bit rel) : 0F 85 → JE/JZ near : 0F 84

➤ **Etapes**:

➤ Repérer le **JNZ** ciblé (adresse + longueur)

➤ Remplacer l'opcode par son **jumeau inversé**



M4 — PATCHING: REDIRECTION VIA JMP COURT

- But : sauter un bloc (ex : fail-path) vers le chemin « success ».
- **JMP short** : opcode EB + disp8 (-128..+127 bytes)
- **JMP near** : E9 + disp32



TP - PATCH LICENCE - DARK SIDE



- Utiliser : une copie de `license_check`
- Localiser le `jne` / `je` utilisé pour vérifier la licence
- Utiliser la méthode de contournement de votre choix
- Générer une version modifiée acceptant n'importe quelle clé de licence (sauf éventuellement la bonne)
- Générer un patch permettant de modifier l'original et faire sauter la vérification

TP - PATCH LICENCE - GOOD SIDE



- Quelles pistes pour protéger le binaire contre ce contournement ?
 - Pour chacune d'entre-elles, déterminer sa facilité de mise en place ainsi que ses limites



MODULE 5 — MALWARE ET VULNÉRABILITÉS



MODULE 5 — MALWARE ET VULNÉRABILITÉS:

OBJECTIFS

- Extraire rapidement comportements & IOC exploitables
- Introduire ROP et les mitigations modernes
- Donner un aperçu firmware (optionnel)



M5 — SÉCURITÉ DU LABO



M5 — SÉCURITÉ DU LABO

➤ VM isolée & snapshots

- Hyperviseur type-2 : VirtualBox/VMware
- **Aucun exécutable** hors VM. *Ne jamais* lancer un sample sur l'hôte
- **Snapshots fréquents** : avant chaque session & après configuration outils
- ISO « outils » en lecture seule ; **pas** de copier/coller bidirectionnel

➤ Partages & transferts

- Dossier **unidirectionnel** (host→guest only). Idéal : export **read-only**
- Alternatif : ISO/IMG montés en RO ; **aucun** montage réseau RW



M5 — SÉCURITÉ DU LABO

➤ Réseau

- Mode NAT (pas de bridge) ; pas d'accès entrant vers la VM
- Blocage egress par défaut ; n'ouvrir que ce que l'on trace
- Option avancée : rediriger le trafic vers une proxy/sinkhole locale (INetSim)

```
# UFW (dans la VM)
sudo ufw default deny outgoing
sudo ufw deny out to any
sudo ufw allow out 53/udp # si DNS vers sinkhole local uniquement
sudo ufw enable
```

➤ Capture

- Lancer wireshark côté VM
- sysdig -w traces/session.scap



M5 — IOC ET COMPORTEMENTS



M5 — IOC ET COMPORTEMENTS

➤ Que chercher ?

- Réseau : domaines/IP, URLs, ports, protocoles
- Fichiers : chemins, extensions, hashes, clés/sel RC4/AES hardcodés
- Persistance : clés Reg / services / crontab / .desktop / launch agents
- Process/Exec : commandes, parent PID, injection (ptrace/WriteProcessMemory)

➤ Commandes utiles

- `sudo sysdig -w trace.scap 'evt.type in (execve,open,connect)'`
- `strace -f -o strace.log ./sample --arg`
- `ltrace -f -o ltrace.log ./sample`



M5 — IOC ET COMPORTEMENTS

➤ Quick wins

- URLs: `strings malware.bin | grep -Eo 'https?://[^\s]+'`
- Noms de domaine : `strings malware.bin | grep -Eoi '([A-Za-z0-9-]+\.[A-Za-z]{2,63})'`



M5 — VULNÉRABILITÉS MÉMOIRE



M5 — VULNÉRABILITÉS MÉMOIRE

- Stack overflow: signature et crash
- Protections: NX, ASLR, canaris
- ROP: gadgets et chaînes (intro)

M5 — STACK OVERFLOW — SIGNATURE & CRASH



➤ Signature typique

- Entrée non bornée (ex : strcpy, gets, scanf("%s"))
- Crash SIGSEGV à l'épilogue (leave; ret) ou sur ret → **RIP/EIP** corrompu
- Motifs répétitifs dans la pile (ex : 0x41414141)

➤ Conséquences :

- Crash / DoS au mieux
- Exécution de commande au pire



M5 — PROTECTIONS — NX, ASLR, CANARIS

➤ NX (no-exec stack)

- Empêche exécution sur la pile → `ret2libc/ROP` nécessaires
- Vérifier : `readelf -lW a.out | grep GNU_STACK` (doit être RW, pas E)

➤ ASLR

- Randomize bases (stack/heap/libc/mmap)
- État : `cat /proc/sys/kernel/randomize_va_space` (2=on)
- Impact : adresses mouvantes → besoin de `leak` ou d'offsets relatifs

➤ Canaris (Stack Protector)

- Sentinelle entre locals et retour, vérifiée en sortie ; overflow → `__stack_chk_fail` (SIGABRT)
- Détection : `readelf -s a.out | egrep '__stack_chk_(fail|guard)'`
- Compiler sans (pour lab) : `-fno-stack-protector`



M5 — ROP — GADGETS & CHÂÎNES (INTRO)

➤ Idée

- Réutiliser **gadgets** se terminant par `ret` pour construire une **chaîne** réalisant une action (ex : `execve`)
- Sources de gadgets : binaire + bibliothèques (libc)

➤ `ROPgadget --binary ./vuln | head`

- `ropper --file ./vuln --search "pop rdi; ret"`

➤ Exemple de ROPChain :

- `[padding] [pop rdi; ret] ["/bin/sh"] [pop rsi; ret] [0] [pop rdx; ret] [0] [ret2libc: system]`

TP FINAL - ANALYSE DE MALWARE ET RAPPORT



- Récupérer sur discord :
 - Le fichier docx contenant consignes + template de rapport
 - Le fichier 7z contenant le malware
- Durée 3h
- Travail individuel



RESSOURCES

➤ Pour se former, se certifier :

- <https://academy.hackthebox.com/catalogue/paths>
- <https://www.root-me.org>
- Gameshell : <https://github.com/phyver/GameShell>

➤ Blogs et veille :

- <https://talkback.sh>
- <https://hackers-arise.com/articles/>
- <https://www.zataz.com>
- <https://thehackernews.com>
- <https://quentin.papelard.pro/blog-cybersecurite>
- <https://vx-underground.org>

➤ Les cheatsheets :

- Tout Linux : <https://www.geeksforgeeks.org/linux-unix/linux-commands-cheat-sheet/>
- Swissky - PayloadAllTheThings : <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master>
- PentestMonkey : <https://pentestmonkey.net/category/cheat-sheet>
- HackTricks : <https://hacktricks.wiki/en/index.html>
- MSFVenom : <https://nitesculucian.github.io/2018/07/25/msfvenom-cheat-sheet/>
- GTFOBins : <https://gtfobins.org/>
- Equivalent windows : <https://lolbas-project.github.io>
- vim : <https://vim.rtorr.com/>



RESSOURCES

➤ Outils

- LinPEAS : <https://linpeas.org>
- LSE : <https://github.com/diego-treitos/linux-smart-enumeration/blob/master/lse.sh>
- Cyberchef (utiliser localement !) : <https://gchq.github.io/CyberChef/>
- Wigle.net (cartographie des éléments wireless) : <https://wigle.net>
- Shodan.io (cartographie internet, recherche de services exposés, etc.) : <https://www.shodan.io>

➤ Youtube

- David Bombal (interview cyber) : <https://www.youtube.com/@davidbombal>
- John Hammond (hacker, démos, malware analysis ...) : https://www.youtube.com/@_JohnHammond
- NetworkChuck (IA, homelab ...) : <https://www.youtube.com/@NetworkChuck>
- TalkingSasquatch (devices de hacking) : <https://www.youtube.com/@TalkingSasquatch>

RESSOURCES



➤ Docus :

- Documentaire « Don't go to the police » :

<https://www.youtube.com/watch?v=0pchn3bkgs8>

- Cas d'école, la chute de yggtorrent par Underscore :

<https://www.youtube.com/watch?v=ULTV6T6jRTw>

- Communication offgrid : https://www.youtube.com/watch?v=UTWrq_nS56A